

# AI Recruitment Dashboard - Project Documentation

## Project Overview

AI Recruitment Dashboard, also called RecruitAI in the application UI, is an AI-powered recruitment platform for analyzing and ranking candidates against a job description. It allows a recruiter to upload or paste a job description, upload multiple PDF resumes, watch the processing progress in real time, review ranked candidates, shortlist strong matches, export reports, and send candidate outreach emails.

The project is split into a React frontend and a FastAPI backend. The frontend provides the recruiter dashboard experience, while the backend handles PDF processing, resume parsing, scoring, persistence, Excel report generation, and email delivery.

## Main User Workflow

1. The recruiter enters a job description manually or uploads a job description PDF.
2. The recruiter uploads candidate resumes as PDF files.
3. The backend validates the uploads and creates an asynchronous analysis job.
4. The frontend displays a live scanning screen while resumes are processed.
5. The backend extracts resume text, using OCR as a fallback for scanned PDFs.
6. Candidate details such as name, email, skills, education, experience, projects, and organizations are extracted.
7. Candidates are scored against the job description.
8. The dashboard displays candidate ranking, match scores, skills, gaps, analytics, and AI-generated insights.
9. Recruiters can shortlist candidates, send emails, schedule calls, add notes, and export Excel reports.

## Technology Stack

### Frontend

- ? React 19
- ? TypeScript
- ? Vite
- ? Tailwind CSS 4
- ? Lucide React icons
- ? xlsx package for spreadsheet-related functionality

### Backend

- ? FastAPI
- ? SQLAlchemy

- ? Pydantic
- ? SQLite by default
- ? PostgreSQL support for Docker and production-style development
- ? PyMuPDF for PDF text extraction
- ? pytesseract and Pillow for OCR
- ? spaCy for NLP support
- ? Sentence Transformers for semantic similarity
- ? scikit-learn and NumPy for scoring support
- ? OpenPyXL for Excel report generation
- ? Ollama integration for local LLM-based resume parsing

## Project Structure

### Frontend Files

- ? 'src/App.tsx': Main application state and screen routing.
- ? 'src/screens/NewAnalysis.tsx': Job description and resume upload screen.
- ? 'src/screens/AiScanning.tsx': Live analysis progress screen.
- ? 'src/screens/Overview.tsx': Recruitment dashboard, KPIs, charts, filters, and candidate lists.
- ? 'src/screens/CandidateAnalysis.tsx': Detailed candidate view.
- ? 'src/screens/ShortlistedCandidates.tsx': Shortlist management and bulk outreach.
- ? 'src/screens/EmailPage.tsx': Email composition and sending UI.
- ? 'src/components/Sidebar.tsx': Main dashboard navigation.
- ? 'src/components/TopBar.tsx': Header actions such as new analysis and export.
- ? 'src/utils/api.ts': Frontend API client.
- ? 'src/data/candidates.ts': Candidate types, conversion helpers, and sample data.

### Backend Files

- ? 'backend/app/main.py': FastAPI app, endpoints, startup database setup, report generation, and routing.
- ? 'backend/app/services.py': Main resume analysis pipeline.
- ? 'backend/app/nlp.py': PDF extraction, OCR, NLP helpers, skill extraction, semantic similarity, name extraction, and scoring helpers.
- ? 'backend/app/models.py': SQLAlchemy database models.
- ? 'backend/app/schemas.py': Pydantic request and response schemas.
- ? 'backend/app/database.py': Database engine and session setup.
- ? 'backend/app/config.py': Application settings and environment configuration.
- ? 'backend/app/email\_service.py': SMTP email sending and email templates.
- ? 'backend/requirements.txt': Python dependencies.
- ? 'docker-compose.yml': PostgreSQL and API development environment.

## Backend Data Model

### Analysis

The 'Analysis' model represents one resume-analysis batch. It stores:

- ? Analysis ID
- ? Job title
- ? Job description
- ? Extracted job requirements
- ? Status such as queued, processing, completed, or failed
- ? Error message if processing fails
- ? Created and completed timestamps
- ? Total resume count
- ? Current processing phase and stage
- ? Current resume filename and candidate name
- ? Progress counters
- ? Related candidates

## Candidate

The 'Candidate' model stores parsed and scored resume data. It includes:

- ? Candidate ID
- ? Analysis ID
- ? Original filename and stored filename
- ? Name and email
- ? Extracted resume text
- ? Skills and missing skills
- ? Extracted entities such as emails, phones, organizations, locations, and experience years
- ? Projects
- ? Education
- ? Experience details
- ? Semantic score
- ? Keyword or skill score
- ? Experience score
- ? Overall score
- ? Recommendation label
- ? AI-generated insight
- ? Manual name-entry flag
- ? Shortlist status
- ? Email sent status and timestamp
- ? Call scheduled status and timestamp
- ? Internal notes

## Resume Analysis Pipeline

The backend analysis starts when the frontend calls 'POST /api/v1/analyses'. The backend creates an analysis record, stores uploaded resume PDFs, and starts a background task.

Processing happens in phases:

1. Text extraction

- PDF files are read using embedded text extraction.
- If a resume is scanned or image-only, OCR is attempted.
- Unreadable resumes are skipped without failing the entire batch unless no resumes can be processed.

2. Job requirement extraction

- The job description is analyzed to identify required and preferred skills.
- Requirements are stored as required, preferred, and combined skill lists.

3. Semantic similarity

- Resume text is compared with the job description using sentence embeddings.
- The configured model is 'all-MiniLM-L6-v2'.

4. Candidate metadata extraction

- The system attempts structured extraction using Ollama.
- If Ollama extraction fails, the backend falls back to rule-based extraction.
- Extracted metadata includes name, email, phone, skills, experience, education, projects, and work history.

5. Skill normalization

- Skills are normalized through a canonical skill mapping.
- This helps match variants such as 'js', 'JavaScript', 'node', 'Node.js', and similar aliases.

6. Score calculation

- Required skill coverage is weighted more heavily than preferred skill coverage.
- Semantic similarity is combined with skill alignment.
- Experience and education alignment are considered.
- A composite score is generated.

7. Insight generation

- The backend creates a structured explanation containing strengths, gaps, skill coverage, experience assessment, and recommendation.

8. Completion

- Candidate records are saved.
- The analysis status becomes completed.
- Candidates above the shortlist threshold are automatically shortlisted.

## Scoring and Recommendations

The scoring system is designed to be explainable. It considers:

- ? Semantic similarity between resume and job description
- ? Required skill coverage

- ? Preferred skill coverage
- ? Detected experience
- ? Education and role alignment

Candidate recommendation labels include:

- ? Strong Match
- ? Good Match
- ? Moderate Match
- ? Weak Match

Candidates with an overall score above 75 are automatically marked as shortlisted.

## Main API Endpoints

### Health

- ? 'GET /health'

Returns service status, upload limits, and OCR availability.

### Analysis

- ? 'POST /api/v1/analyses'
- ? 'GET /api/v1/analyses'
- ? 'GET /api/v1/analyses/{analysis\_id}'
- ? 'GET /api/v1/analyses/{analysis\_id}/status'
- ? 'GET /api/v1/analyses/{analysis\_id}/report.xlsx'

These endpoints create analyses, list analyses, fetch results, poll processing status, and download full Excel reports.

### Resume and Job Description

- ? 'GET /api/v1/candidates/{candidate\_id}/resume'
- ? 'POST /api/v1/job-descriptions/extract'

These endpoints serve original candidate resumes and extract text from uploaded job description PDFs.

### Shortlisting and Filtering

- ? 'POST /api/v1/candidates/{candidate\_id}/shortlist'
- ? 'GET /api/v1/analyses/{analysis\_id}/shortlisted'
- ? 'POST /api/v1/analyses/{analysis\_id}/filtered-candidates'

? 'GET /api/v1/analyses/{analysis\_id}/shortlisted-report.xlsx'  
? 'GET /api/v1/candidates/strong-matches'

These endpoints manage shortlisted candidates, candidate filters, and shortlisted reports.

## Email and Outreach

? 'POST /api/v1/candidates/send-email'  
? 'POST /api/v1/candidates/{candidate\_id}/schedule-call'

These endpoints send candidate emails and track scheduled calls.

## Key Features

? Bulk resume PDF upload  
? Job description text input  
? Job description PDF upload and extraction  
? Resume PDF validation  
? OCR fallback for scanned resumes  
? Real-time analysis progress tracking  
? AI and NLP-based resume analysis  
? Skill extraction and normalization  
? Required and preferred skill matching  
? Candidate ranking  
? Match score breakdown  
? Missing skill detection  
? Candidate profile pages  
? AI-generated candidate insights  
? Shortlist management  
? Email outreach with templates  
? Call scheduling status  
? Internal notes  
? Full Excel report export  
? Shortlisted-only Excel report export  
? Original resume download endpoint  
? SQLite local database support  
? PostgreSQL support through Docker Compose

## Configuration

The backend settings are defined in 'backend/app/config.py' and can be overridden using environment variables.

Important settings include:

? 'DATABASE\_URL'

? 'UPLOAD\_DIR'  
? 'MAX\_RESUMES\_PER\_ANALYSIS'  
? 'MAX\_FILE\_SIZE\_BYTES'  
? 'CORS\_ORIGINS'  
? 'EMBEDDING\_MODEL'  
? 'OCR\_DPI'  
? 'TESSERACT\_CMD'  
? 'OLLAMA\_BASE\_URL'  
? 'OLLAMA\_MODEL'  
? 'OLLAMA\_TIMEOUT'  
? 'SMTP\_SERVER'  
? 'SMTP\_PORT'  
? 'SMTP\_USER'  
? 'SMTP\_PASSWORD'  
? 'SENDER\_EMAIL'  
? 'SENDER\_NAME'

## Running the Project

### Frontend

```
npm install  
npm run dev
```

The frontend runs through Vite, usually at:

<http://localhost:5173>

### Backend

```
cd backend  
python3 -m venv .venv  
source .venv/bin/activate  
pip install -r requirements.txt  
python -m spacy download en_core_web_sm  
uvicorn app.main:app --reload --port 8000
```

The backend API runs at:

`http://localhost:8000`

Swagger documentation is available at:

`http://localhost:8000/docs`

## Docker Development

`docker compose up --build`

This starts PostgreSQL and the backend API.

## Email Setup

To send emails, configure SMTP settings in the backend '.env' file.

Example Gmail configuration:

```
SMTP_SERVER=smtp.gmail.com
SMTP_PORT=587
SMTP_USER=your-email@gmail.com
SMTP_PASSWORD=your-app-password
SENDER_EMAIL=your-email@gmail.com
SENDER_NAME=RecruitAI
```

For Gmail, an app password is usually required.

## Upload Constraints

- ? Maximum resumes per analysis: 60
- ? Maximum file size per resume: 5 MB
- ? Resume format: PDF only
- ? Job description PDF upload is supported

## Reports

The system generates Excel reports using OpenPyXL.

The full report includes:

- ? Summary sheet
- ? Candidate ranking sheet
- ? Candidate details sheet
- ? Scores
- ? Contact information
- ? Education
- ? Skills
- ? Missing skills
- ? Projects
- ? Organizations
- ? Locations
- ? AI insight

The shortlisted report includes only shortlisted candidates and adds workflow fields such as email sent, call scheduled, and notes.

## Current Development Notes

At the time this documentation was generated, the repository had existing local modifications in:

- ? 'backend/app/email\_service.py'
- ? 'backend/app/main.py'
- ? 'src/screens/Overview.tsx'

There was also a new test file:

- ? 'backend/test\_email\_service.py'

These appear to be active local changes and were not modified while creating this document.

## Conclusion

AI Recruitment Dashboard is a full-stack recruiting tool that combines document processing, NLP, semantic matching, structured candidate scoring, recruiter workflow management, and email outreach. It is suitable for local development with SQLite, Docker-based development with PostgreSQL, and can be extended toward production with stronger authentication, background job infrastructure, retention policies, and deployment hardening.